

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

CODING CHALLENGE

Duration: 2hrs

1. Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making. **(5 marks)**

2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability,

reward function, and discount factor. **(5 marks)**

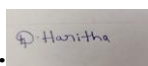
3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation. **(5 marks)**

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ϵ -greedy and softmax approaches in balancing learning efficiency. (5 marks)
5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies. (5 marks)
6. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance. (5 marks)
7. Analyze the influence of the discount factor (γ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks. (5 marks)
8. Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations. (5 marks)
9. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior. (5 marks)
- 10 Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications. (5 marks)

Code of Conduct:

*I **Dasari Haritha(192425254)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



CODING CHALLENGE

Question 1: Reinforcement Learning (RL) Framework

PROBLEM STATEMENT

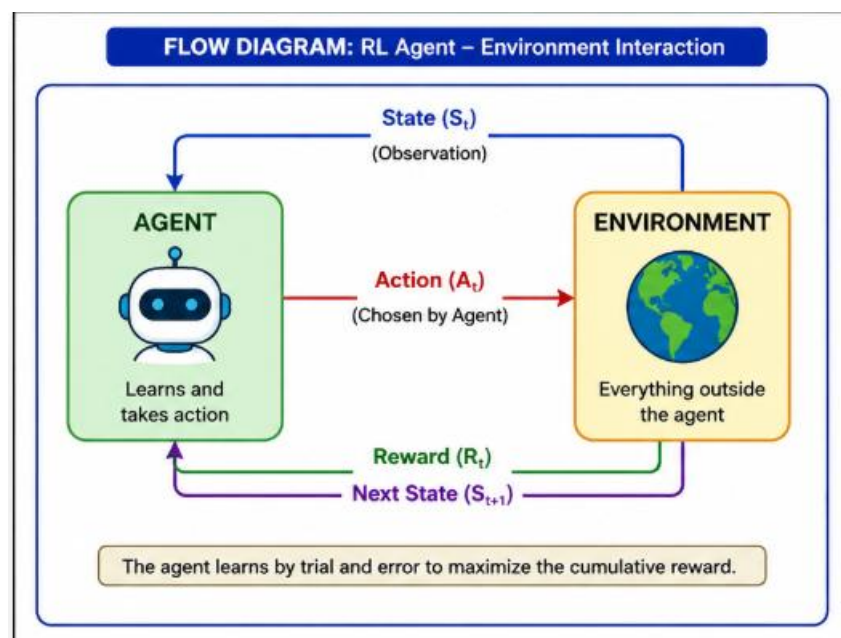
Design a Reinforcement Learning (RL) framework where an agent interacts with an environment, learns from rewards and penalties, and chooses actions that maximize the cumulative reward.

PROBLEM SOLUTION

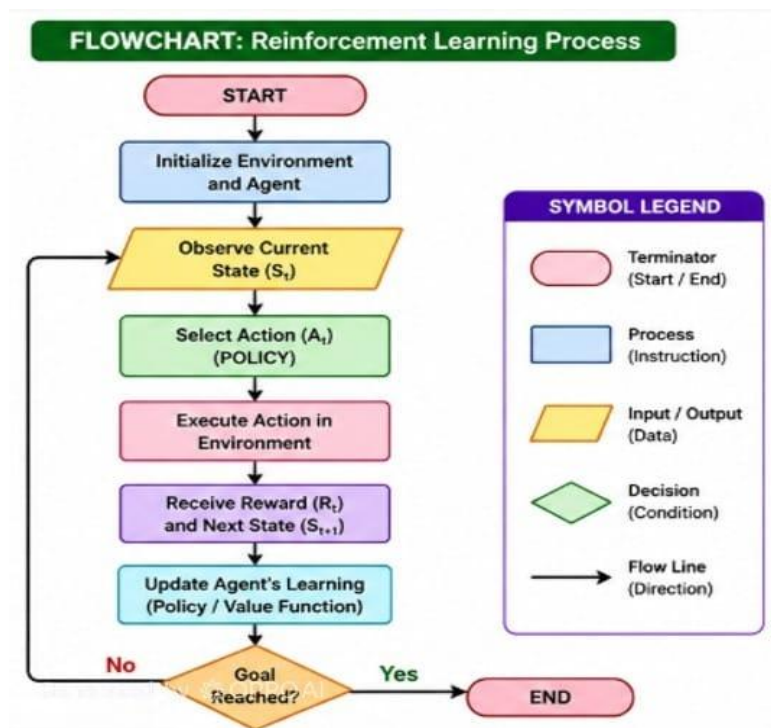
The RL framework solves the problem through continuous interaction between the Agent and the Environment.

1. The environment provides the current state.
2. The agent observes the state.
3. The agent selects an action.
4. The environment executes the action.
5. The environment returns a reward and the next state.
6. The agent updates its learning policy based on the reward.
7. The process repeats until the goal is achieved.

FLOW DIAGRAM



FLOWCHART



PYTHON PROGRAM

```
states = ["Start", "Middle", "Goal"]
rewards = [5, 10]
current_state = 0
total_reward = 0
print("Reinforcement Learning Simulation")
print("-----")
while current_state < len(states) - 1:
    print("Current State :", states[current_state])
    action = "Move Forward"
    print("Action :", action)
    reward = rewards[current_state]
    print("Reward :", reward)
    total_reward += reward
    current_state += 1
    print("Next State :", states[current_state])
    print()
```

```
print("Goal Reached!")
print("Total Reward :", total_reward)
```

Sample Output

Current State : Start
Action : Move Forward
Reward : 5
Next State : Middle
Current State : Middle
Action : Move Forward
Reward : 10
Next State : Goal
Goal Reached!
Total Reward : 15

QUESTION 2 — Markov Decision Process (MDP)

PROBLEM STATEMENT

Design a **Markov Decision Process (MDP)** to represent a decision-making problem in Reinforcement Learning, where an agent selects actions in different states, moves to new states according to transition probabilities, and receives rewards. The objective is to select actions that maximize the long-term cumulative reward.

PROBLEM SOLUTION

An MDP models sequential decision-making using five components:

Where:

- $S \rightarrow$ Set of states
- $A \rightarrow$ Set of actions
- $P(s'|s,a) \rightarrow$ Transition probability
- $R(s,a) \rightarrow$ Reward function
- $\gamma \rightarrow$ Discount factor

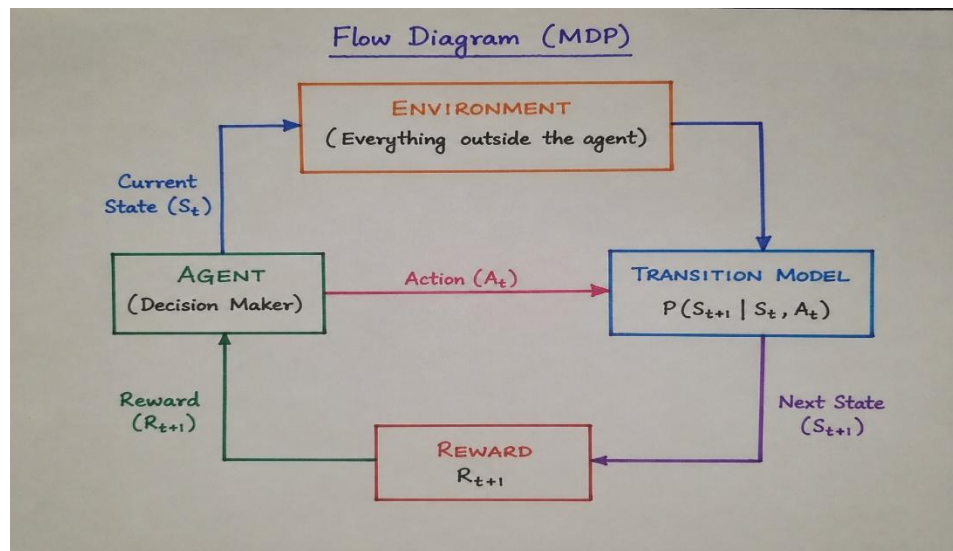
The agent observes the current state, chooses an action, receives a reward, and moves to the next state. This process continues until the desired goal is reached.

EXAMPLE

Consider a student travelling from **Home** $\rightarrow$ **College**.

- **State:** Home
- **Actions:** Walk, Bus
- **Next State:** College
- **Reward:** Time/cost-based reward
- **Discount factor:** Determines the importance of future rewards.

FLOW DIAGRAM



PYTHON PROGRAM

```
states = ["Home", "College"]  
  
# Actions  
actions = ["Walk", "Bus"]  
  
# Transition probabilities  
transition_probability = {  
    ("Home", "Walk"): 1.0,  
    ("Home", "Bus"): 1.0  
}  
  
# Reward function  
rewards = {  
    ("Home", "Walk"): 5,  
    ("Home", "Bus"): 10  
}
```

```

gamma = 0.9
# Initial state
current_state = "Home"
# Select action
action = "Bus"
# Find next state
next_state = "College"
# Get reward
reward = rewards[(current_state, action)]
print("===== MARKOV DECISION PROCESS =====")
print("States :", states)
print("Actions :", actions)
print("Discount Factor :", gamma)
print("\nCurrent State :", current_state)
print("Selected Action :", action)
print("Transition Probability :",
      transition_probability[(current_state, action)])
print("Reward Received :", reward)
print("Next State :", next_state)
print("\nGoal Reached : Yes")

```

Output

```

===== MARKOV DECISION PROCESS =====
States : ['Home', 'College']
Actions : ['Walk', 'Bus']
Discount Factor : 0.9
Current State : Home
Selected Action : Bus
Transition Probability : 1.0
Reward Received : 10
Next State : College
Goal Reached : Yes

```

QUESTION 3 – Policy and Value Functions Using Bellman Equation

PROBLEM STATEMENT

Develop a Reinforcement Learning solution that uses a **policy function**, **state-value function**, and **action-value function** to evaluate the long-term benefits of actions using the **Bellman Equation**.

PROBLEM SOLUTION

In Reinforcement Learning, the **policy** determines which action the agent should take. The **value function** estimates how good a state or action is based on expected future rewards.

The Bellman Equation updates the value of a state using:

where:

- = Value of current state
- = Immediate reward
- = Discount factor
- = Value of next state

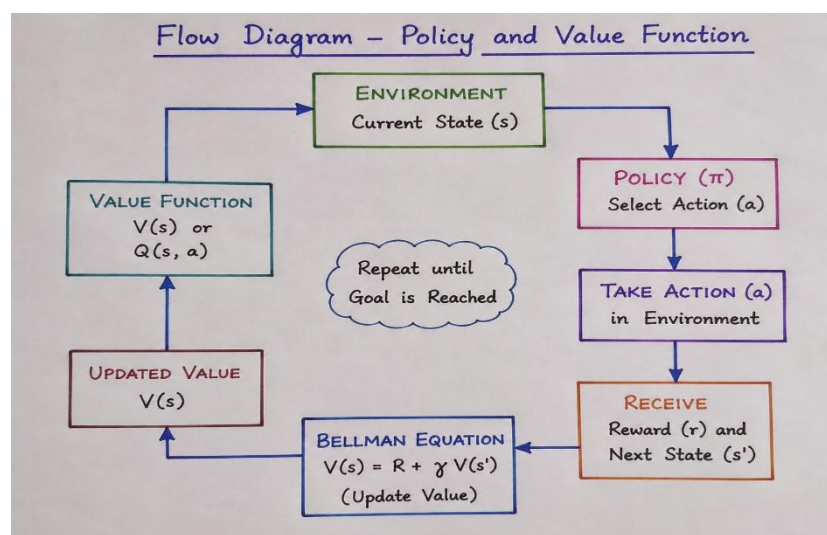
For example, if:

- Immediate reward = 10
- Discount factor = 0.9
- Future state value = 20

Then:

Therefore, the updated value of the current state is **28**.

FLOW DIAGRAM



PYTHON PROGRAM

Policy and Value Function using Bellman Equation

Discount factor

gamma = 0.9

Immediate reward

reward = 10

Value of the next state

future_value = 20

Bellman Equation

current_value = reward + gamma * future_value

print("=== Policy and Value Function ===")

print("Immediate Reward :", reward)

print("Discount Factor :", gamma)

print("Future State Value :", future_value)

print("\nBellman Equation:")

print("V(s) = R + gamma * V(s')")

print("\nUpdated State Value :", current_value)

OUTPUT

=== Policy and Value Function ===

Immediate Reward : 10

Discount Factor : 0.9

Future State Value : 20

Bellman Equation:

V(s) = R + gamma * V(s')

Updated State Value : 28.0

QUESTION 4 – Exploration–Exploitation Trade-off

PROBLEM STATEMENT

Develop a Reinforcement Learning solution that balances **exploration** of new actions and **exploitation** of the best-known action using the **ϵ -greedy strategy**. The agent should select actions based on a given exploration probability and receive rewards from the environment.

PROBLEM SOLUTION

The agent faces two choices:

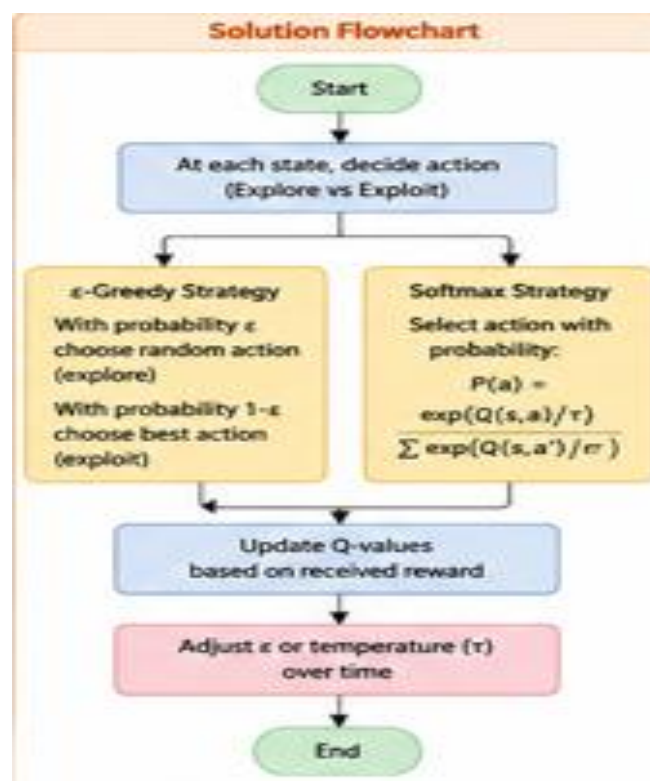
- **Exploration:** Try a new action to discover whether it gives a better reward.
- **Exploitation:** Select the action that currently has the highest known reward.

In the **ϵ -greedy method**:

For example, if:

then the agent explores with probability **20%** and exploits the best-known action with probability **80%**.

FLOW DIAGRAM



PYTHON PROGRAM

```
import random
# Exploration probability
epsilon = 0.2
# Available actions
actions = ["Left", "Right", "Up", "Down"]
# Known reward values for actions
q_values = {
    "Left": 2,
    "Right": 8,
    "Up": 4,
    "Down": 3
}
# Generate random number
random_number = random.random()
print("=== Epsilon-Greedy Strategy ===")
print("Epsilon :", epsilon)
print("Q-Values :", q_values)
if random_number < epsilon:
    # Exploration
    action = random.choice(actions)
    print("\nMode : Exploration")
    print("Random Action :", action)
else:
    # Exploitation
    action = max(q_values, key=q_values.get)
    print("\nMode : Exploitation")
    print("Best Action :", action)
print("Reward :", q_values[action])
```

OUTPUT

Example 1 – Exploitation

=== Epsilon-Greedy Strategy ===

Epsilon : 0.2

Q-Values : {'Left': 2, 'Right': 8, 'Up': 4, 'Down': 3}

Mode : Exploitation

Best Action : Right

Reward : 8

Example 2 – Exploration

=== Epsilon-Greedy Strategy ===

Epsilon : 0.2

Q-Values : {'Left': 2, 'Right': 8, 'Up': 4, 'Down': 3}

Mode : Exploration

Random Action : Up

Reward : 4

QUESTION 8 – Model-Free RL vs Model-Based RL

Problem Statement

Develop a Reinforcement Learning solution that demonstrates how an agent can make decisions in a dynamic environment using **model-free reinforcement learning**, and compare it with model-based reinforcement learning.

Problem Solution

Reinforcement Learning can be divided into two major approaches:

Model-Free RL

The agent learns directly from experience without knowing the environment's transition model.

Example:

Q-learning

The agent learns:

by repeatedly taking actions and receiving rewards.

Model-Based RL

The agent learns or uses a model of the environment:

It can predict the result of actions before actually performing them.

Comparison

Model-Free RL

Does not require environment model

Learns through experience

Simple to implement

Usually needs more samples

Q-learning, SARSA

Model-Based RL

Uses environment model

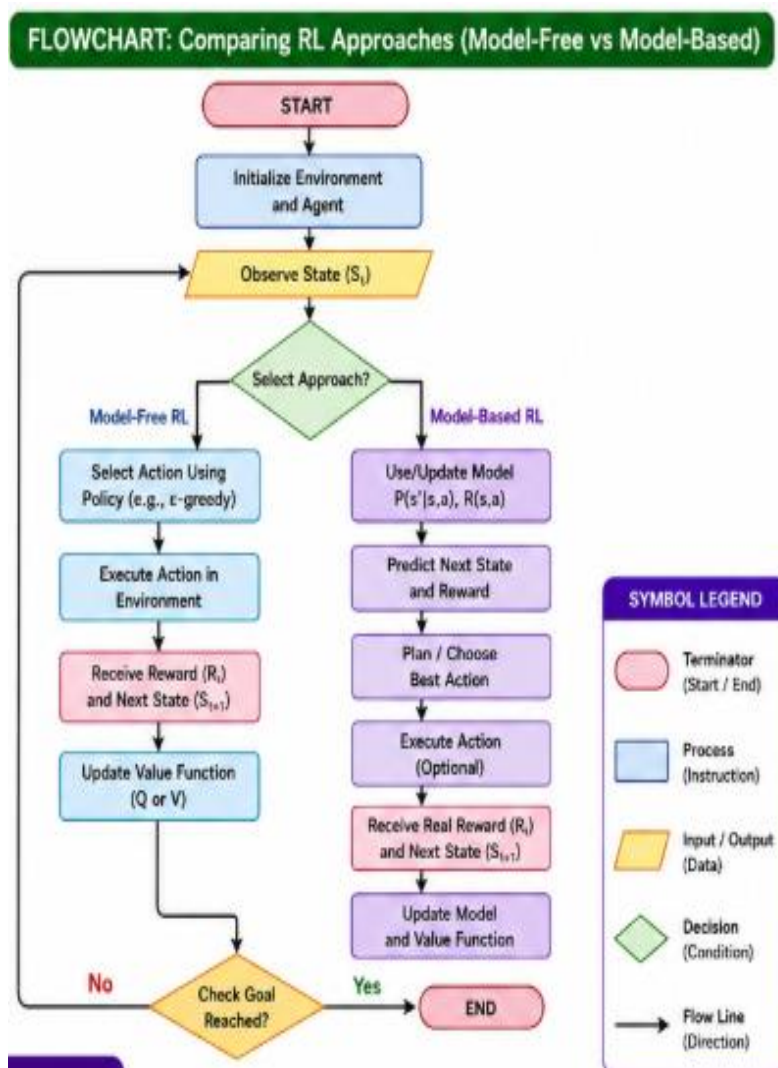
Predicts future outcomes

More complex

More sample efficient

Planning, Dyna-Q

Flowchart



Python Program

```
# Model-Free Reinforcement Learning
# Simple Q-Learning Example
states = ["Start", "Middle", "Goal"]
actions = ["Move"]
# Q-values
Q = {
    ("Start", "Move"): 0,
    ("Middle", "Move"): 0
}
alpha = 0.5    # Learning rate
gamma = 0.9    # Discount factor
rewards = {
    "Start": 5,
    "Middle": 10
}
print("=== Model-Free Reinforcement Learning ===")
for episode in range(1, 6):
    state = "Start"
    # First action
    action = "Move"
    next_state = "Middle"
    reward = rewards[state]
    old_q = Q[(state, action)]
    next_q = Q[(next_state, action)]
    Q[(state, action)] = old_q + alpha * (
        reward + gamma * next_q - old_q
    )
    # Second action
    state = "Middle"
    action = "Move"
    next_state = "Goal"
```

```

reward = rewards[state]
old_q = Q[(state, action)]
# Goal has no future value
next_q = 0
Q[(state, action)] = old_q + alpha * (
    reward + gamma * next_q - old_q
)
print("Episode", episode)
print("Q(Start, Move) =", round(Q[("Start", "Move")], 2))
print("Q(Middle, Move) =", round(Q[("Middle", "Move")], 2))
print()

```

Output

=== Model-Free Reinforcement Learning ===

Episode 1

Q(Start, Move) = 2.5

Q(Middle, Move) = 5.0

Episode 2

Q(Start, Move) = 6.0

Q(Middle, Move) = 7.5

Episode 3

Q(Start, Move) = 8.75

Q(Middle, Move) = 8.75

Episode 4

Q(Start, Move) = 10.25

Q(Middle, Move) = 9.38

Episode 5

Q(Start, Move) = 11.06

Q(Middle, Move) = 9.69

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problemsolving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides wellstructured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately